

ActInf ModelStream #003.1 ~ Ozan Catal & Tim Verbelen

ModelStream | Ozan Catal & Tim Verbelen | 1:27:32

hello

everyone welcome to actin flab this

is actin flab model stream number 3.1

it's may 27th 2021

and we're here with tim verbellen and

ozone catal

today we're going to have a model stream

on some of their recent work

on learning generative state-space

models for active inference

we're gonna have a presentation followed

by some time

for question and answer so please feel

free to write

any questions in the chat and we will

get to them

in the conversation so thanks again to

tim and ozon for

joining us today we're really looking

forward to what you have to share

so please take it away and thanks again

so thank you daniel for having us um

so i'm tim verde and together with uh

rosencattel we will

talk a bit on our paper on learning

alternative state space models for

active inference so we are both

researchers at

imek kent university in belgium

if you want to know more about what we

do you can

read about it on our blog or follow us

on twitter

under under at the smart robots

so before we

dive in maybe a short introduction on on
what we do what our goal is
so basically we want to build
intelligent robots so in our lab
here in gans we have a really big
pretty pretty large space where we have
some room for robot manipulators on the
one hand but also
driving and flying robots on the other
hand
and we typically attach all kinds of
sensors on these things
and then we want to process the sensor
information
and infer some useful actions for these
things and
of course active inference is a cool
methodology
to try out and to
further investigate and it's in this
context that basically this this work
is being done
so why would you bother learning the
state space so if you look at
active inference papers from the
recent years then typically you will
find a figure
like the ones that are on the slide
so you have this figure of what is the
kind of environments
that that you're that you're modeling
that you're investigating
and then a whole description on how the
model should look like
so you define the state space and then
possibly or in the case of discrete
state spaces you you
define the the so-called a b c and d

matrix that's
uh that defines the the likelihood model
so what what is the
likelihood to see a certain observation
when you are in certain state
or whether the transition models uh how
do you transition from
one state to another and so forth
um and recently while we were doing this
research
research we saw some other people also
thinking about
yeah um can we do more
learning using these these kind of novel
deep learning techniques
in these uh in these active inference
methods so yet for example kai will
suffer
um who said uh you proposed to to
basically learn kind of a state space
for
for the mountain car but then still he
needed to
explicitly encode some of the of the
environment information
in in the state vector
there was also some work from from from
baron millage
who basically did some learning
a bit more on learning the policy rather
than
the state space so um basically used um
work of decision process with small
state spaces where you could basically
just
deal with the the raw state space
the the observations that the
environment gave you where

were basically suited as a state space
and he could then
he would then investigate how to learn
the
actions given given these states but so
our work is basically on what if you
don't know the state space what if your
observations are
high dimensional and you cannot really
use this directly as your state space
how do you define
the model how you can come up with it
and maybe as a concrete example suppose
we have
one of our driving robots in the lab so
this this is a first-person view of the
robots
this is the kind of observation that you
get is just a square of pixels
and then yeah what's what is your state
space in this case
it might be an xy position on the map
that might be something relevant for a
robot
as a state it might also be uh
watch out you're approaching a cable
getter so there might be a
that might also be something useful
maybe the items that are stored in your
wrecks are
are relevant for for whatever task you
have so that might also be part of the
state space
maybe there are human workers that walk
around that you also need to model these
so
depending on your robot and the use case
you have it does get hands and what can

happen in the environment
it becomes non-trivial to define
kind of a concise set of states that you
want to track
let alone that you need some model
that's that's
that models this like how do you get
from pixels to
your expected position so there are
certain methods
uh in robotics that's to do
part of these things but none of them
really just figure out what is the
complete
uh state space that you need for for
whatever you want to do
and especially not in kind of an
active inference setting
so before we go into our methods i'll
briefly
talk about active inference probably
most of you are already familiar with
the topic but i'll just
rehearse some of the stuff even if it's
just to get accustomed to our way of
of notation let's say because everybody
has his own
notation and it at least it will
put us all in on the same page for for
an expert
so um it all starts with having an agent
that needs to interact
with the environment and the agent is
assumed to be kind of separated
from the environments from the so-called
markov blankets
so on the one hand the agents can come
from actions

these actions these will impact the environment which is kind of a generative process that has some some hidden states and that provide you given your action it provides you with some sensory observation so in this case an action could be for the robot to drive around and put some currents on on the motors and observation could be some pixels from from the camera and then the goal of the agent is to build a generative model where he has his own uh he builds his own state space basically he derives how actions um have effects on on the states and how these states then then would generate these observations so the question is how can we build a generative model that actually comes up automatically with kind of a proper state space to learn this model so basically the generative model then looks like a probably be a partially observable markov decision process so it just means that we assume that that's the state at a given time step it only depends on the previous states and the action that's that you were doing and each state gives gives rise to a certain observation and so then we end up with the the so-called free energy principle that

basically what we want to do
is going to build a model uh that's that
maximizes log evidence so here
you see the formula for the free energy
and you can unpack it in in several ways
so the first uh uh way is basically by
by
stating that minimizing free energy is
actually
uh maximizing the uh
evidence lower bound let's say and if
you're
uh at this free energy minimum then
basically you have your approximate
posterior that this is actually equals
to your or
very close to your your troops but then
the
the third line is basically what we
typically use
for optimizing our model which is
basically on the one hand
a complexity term you want to
have your your states explain
explain the way as simple as possible
whereas you want to have the highest
accuracy to predict
whatever you're observing so in this
case our
variational approximate posterior is
something that
maps observations and actions into
inferring your current your current
state
but what about the future so in in the
past you basically knew
which actions you were doing and you you
saw the observations

uh that came in so you could kind of
infer which states would have given rise
to the observations so
that's basically the priority of the
past but then in the future
things change a bit because now you
don't know what you will observe
and also you have to select the actions
you will do
so the actions we will basically
denote them as being generated by
so-called policy
but in in this case the policy is
basically just
yeah what is a certain sequence
of actions basically um
and you now have to create some
expectations
not only on future states but also on uh
future observations so what
which states do i think i will visit by
doing some actions
but also what kind of observations do i
think that i will see if i visit these
states so then
the free energy becomes the so-called
expected free energy
and again you can unpack this
this thing um as follows
so basically what happens is again you
have
on the one hand your uh the log
probability of your
uh steer model minus log probability of
your
um uh of your gender
model uh and now the difference is that
um that you don't know yet the

observations in the future
so yeah you conditioned it on the policy
so everything will depend on the
actions you will do and you also have to
take the expectation over all kinds of
of
outcomes you expect and then going to
the second line you basically
make the move that after
if you if your model is sufficiently
trained then basically
um your

[Music]

your true your approximate posterior
will be very close to the
the true posterior and this then
basically allows you to
uh to rewrite the final term well i
i see now that i uh that i
wrote it in in a different way but you
can you can rewrite this
either as an information game on
and moving towards preferred outcomes
but you can also rewrite the way read it
here
where you basically state okay at some
point in the future
i will visit some states so this is the
sloppy of s
esta given the policy and you basically
replace this
by by some prior belief that regardless
which
policy i'm actually choosing i believe
that
some point in the future i will realize
my preferences so that's why you lose
the

the conditioning on the policy uh
in in the in the last equation so this
then basically becomes a KL divergence
that says
okay i i want to find the policies that
actually bring me close to the states
that i
that i prefer that i like to be into and
then on the other hand you have this
ambiguity term that says okay what i
don't like is to visit states that can
give me like any observation
that i cannot predict so i don't like
these
ambiguous states where i don't really
understand basically what's
happening
so that that brings us to the
whole active inference scheme so at each
time step
you basically evaluate your expected
free energy
for each of your policies and you then
basically assume that you will choose
the policy
that will actually minimize your
expected free energy so you
you take the softmax over uh minus g
and you also weigh this with this gamma
parameter which is just like the
precision so if you have a high
precision
then basically you will you will have a
very big
output of a soft max function so the the
softmax will basically
become a max so you then
definitely choose the policy with with

lowest

expected free energy if you relax the
the precision a bit then basically you
you

allow for some or some more randomness
in the system and then basically you end
up inferring the next action

according to what is the the the next
action that i should

take according to this policy but so
when the forward follows

the actual the action selection given
the policy is basically

just a deterministic mapping so the
policies are just

like certain sequence of actions so
given

once you choose the policy the the
action is basically fixed but

in theory you could also make this a
probabilistic mapping

so let's now go to the

the real interesting parts of the work
is is how how do we

learn state spaces with deep neural
networks in such an

an active inference key

well basically

we have two components one is the
generative model that

i rehearse from the first

slide and we have our approximate
posterior model

and basically you can see that there are
three

parts in these equations and the first
part

is basically a transition model so

it's outputs the probability of
your state given your previous states
and the action you did in that state
second part oh yeah and also the
the initial state is basically uh also
part of the transition model but then
you just
provide with a zero action for example
just to bootstrap it
the second part is under accurate model
so given state you want to have a model
that
that predicts what kind of observation
you will you will
see and then finally you have the
posterior model that's given the
previous state
action and your current observation also
has to predict
um which state your also has to infer
which state you're in
so basically the posterior model has to
come up with the same thing as a
transition model but in addition
it has access to your circulation
basically
and again similar to a transition model
to bootstrap this for the initial
observation you don't have
any action but it can be modeled by by
the same thing
as all these these three components we
basically instance the ids
as deep neural nets where
[Music]
they have to basically be trained
to come up with these three
modes so if we

uh put it in a schematic we basically have something like this so given your state an action from the previous time step and the observation from the current time step you on the one hand provides the previous state and action to your transition model which will then output the distribution over the current state and in this case this distribution is basically modeled as a multivariate gaussian distribution so basically the output of the neural nets will be the means and the standard deviations of a gaussian and now you basically use this distribution to then generate samples for the next time step so the posterior model gets also as input to the state and the action from the previous time step but also in addition the current observation and it also does a prediction on the the current time step and after sampling you basically have the likelihood model that can then uh generate a prediction of the outcome and then this state is put into the next time step and this the story repeats so as i already mentioned the output of each of our neural nets is basically some mean and standard deviation of uh

of a multivariate gaussian with
um with the diagonal
covariance matrix and we use the
reparameterization trick to generate
samples so basically
we generate sample by uh having the the
means
and then adds the standard deviation
times
some standard normal noise and this
basically allows you to
back propagates gradients all the way
through
this neural net also if you if you draw
samples from from the models
and then of course for the next time
step you just propagate the sample and
then the story repeats basically
so creating such a model is then
basically
first collecting the data sets of
action observation sequences so you
you take your your your agent you either
let it randomly generate sequences in
the environment or
in case of a real robot for example you
you drive it around yourself
in the environments while you record the
actions and the observations
and while the models aren't converged
you sample some subsequences from your
data sets
you um you estimate the
states that are visited and you
reconstruct
all the observations and then you
basically back propagates the free
energy loss so

this is the the same formula as the free energy in the start so on the one hand you have the kl divergence between the output of your posterior neural nets and your prior neural nets and on the other on the other hand you have this reconstruction error that basically scores how how good you're reconstructing the the actual service and using this loss function you just update the parameters of your real nets and you basically build a model that is able on the one hand to infer your current state given your observation that is able to generate new observations if you know which state you're in you can reconstruct these observations but also you have a transition model that you can then just use to in the state space plan ahead what would happen if i if i would do this action and it will give you a distribution over over the next state basically so then we come to uh the planning part so once you have this model how can you use it to let your agent do useful useful useful things so there we will use monte carlo sampling so one of the things uh one of the limitations of these kind of architectures

is that basically we only approximate the distribution for the next state so given the sample from uh from the previous state and given the action we approximate the distribution for the next stage but then we sample during training as well so we will never have like a good distribution for let's say 10 steps ahead so to predict them distributions for further in the future we we approximate these by doing multiple sampling so for each of the policies we want to be uh we want to evaluate the sample and trajectories that uh that all do the same actions but that's due to the sampling might give us different results in terms of future states and observations so for each time step we then have a bunch of states and a bunch of predicted observations we then fit a gaussian distribution using the sample means and variances and then we estimate the expected free energy as follows so on the one hand we have this scale divergence if you remember that scores how good your distribution of states is according to some prior uh some prior preferences of states so you use this this

this normal this is disgusting
distribution to calculate the gel
divergence with respect to these priors
now on the other hand you have this
entropy term
that scores the the ambiguity of these
states
so here we basically use then the
entropy of
the observations uh generated
in our trajectories and we also added a
scale factor
row to just kind of weigh two terms
uh from one another
which allows you to make an agent
more risky let's say if it's it's more
weight on
realizing preferences early whether you
could also
make a more cautious agent let's say
that's that's really
does not want to end up in some
ambiguous states and crucially here
we also have kind of this recursive
notion in here so
we we do this rollout for k time steps
ahead
and then for the future you could kind
of recursively
look further into the future let's say
and and aggregate
also expectancy energies from from that
point
and make it a bit more clear um
let's try to visualize it a bit better
so suppose at some current time step d
you're in the states
then basically what you do is you use

your transition model to
say okay given that i follow or c_1
um what will be the the next
state and you draw a sample from the
distribution so this is then
 s_d plus one by following paul c_1
and this thing you can repeat for k
times so we
we put in this this k to just have some
kind of
um some force grading of actions
say for example in the case of a driving
robot
it might not make sense to switch action
every uh 10 milliseconds let's say so if
you
if you decide to drive forward you want
to keep on driving forward for at least
some time so that's basically what
where this this k comes from so you
follow the same
uh policy for for some time steps
and you basically sample this end time
so
you you um you repeat
this this process end time so you have
now at each time step and samples for
the kind of states you you think you
will visit after following
the first policy and then if you have a
second policy you can do the same thing
for that policy uh so in this case um
we only consider two potential policies
for instance uh go left or go right
let's say
[Music]
and then of at this point so at
time step t plus k we can basically

repeat

this procedure and say okay what if what

if after k time steps i

switch policy and maybe maybe i first

have to go left and

and before turning right

or maybe i have to turn left twice

so then basically you repeat this

process and this you can basically

keep on adding to the search tree for as

long as

your computation power allows you let's

say

so then how do we calculate the expected

free energy from this

so we we take our uh our formula

so basically what we do is at each time

step

we first use likelihood model to also

predict

what are the observations that i expect

in these states

and you then um look at all your state

samples from a certain time step

and this then gives you this this uh

approximate

gaussian distribution for both the

states and

the observations so you plug these into

the formula

and this then gives you kind of a number

that says this is the

expected free energy if i think i will

i am in states t plus k and i will

follow along with policy

one from uh thereafter and then

basically you can

do this for each each of these sub

branches

and this is then where the recursion

comes in so then we basically

state well i assume that's my active

entrance agent also

at at that time step will

most likely choose the policy with the

the the lowest free energy so basically

we combine

uh the the the free energy for policy

one and

policy two according to uh the rates

that that are provided

using this softmax function so if one of

the

if policy one has a very very low um

expected free energy compared to pulse

c2

then basically only the expected free

energy of policy one will be added

um thereafter if they are kind of the

same

then they will both have kind of a

0.5 weight or if the other one is the

clear winner then it will mainly

be the contribution of this thing but so

we basically combine these

assuming that your agent will at that

point also select

the the policy with the the least free

energy

so now we basically have an estimate of

what will be the expected free energy

for the future given that we are at time

step d plus three

and then we can further uh go up to

three

and so then for um for each branch again

we will use the first part of the
formula to
estimate these these gaussians uh to
calculate the gale divergence and the
entropy term
and then add this together with what we
already had for the
for the the the the remaining part of
the tree basically
and this is uh this is very similar i
think to what
uh uh call fristen uh proposes
sophisticated
uh inference paper so basically that's
at each step in the future you you kind
of
only consider the the free energy of the
branches
that you think will basically be the the
best ones
so in the end you'll end up with an
expected free energy for
both of your policies and you then
select
the best one and start acting
accordingly so
i'll now turn to ozon who will give some
more details on the various
experiments that we did that hopefully
give some more insights on how this
can work in practice so take it away
yeah hi um
i have some echo on my um
maybe it's resolved now yeah sounds fine
thank you
okay um so i'll go briefly
through our initial experiences uh
experiments so

that these are the experiments we did in
like the past one or two years
and so one of the first experiments we
did and reported on was the mountain car
which is a fairly well-known benchmark
i think
so the goal is to have this under
actuated car so it has not does not have
enough power to reach the top of
the hand mountain but you want to reach
it anyway
so ideally your agent should learn here
that it should first go back there to
the left
to gain momentum to gain enough momentum
so it can
climb the steeper mountain on the right
and
even though this is such a low
dimensional problem it only has
positions and velocities
it is actually a very interesting
benchmark to
experiment upon because there's no
greedy solution so any agent that is
just wants to realize its preferences
immediately will fail at this because it
will always
fail to drive up the mountain
um now for our experiments we made it a
bit more difficult
we made the fully observable mountain
car partially observable
so what we did was we omitted the
velocity information
and only provided some noisy estimates
of the actual position
so the agent actually had to learn two

things it had to learn first of all what
its
precise position was and then also how
this
position relates to the momentum by
its uh velocity and then if we take
if we start from the setup and we train
the model that
tim explained a bit earlier um
in its most basic form you can actually
learn
a state space that closely mimics these
physical constraints of the system
by minimizing energy so if you look at
the
right utmost figure you'll see that
um you see this sinusoidal uh
state space dimensions and these
closely mimic the actual observations of
the real world
in the lower image so as you can see in
the lower image we have the
ground truth in green which is an actual
trajectory through
its position of the cards and then also
our state transition model estimate so
our prior estimates on the position
without observing and then also in the
blue line which is a bit harder to see
you can see the same estimate but now
corrected with the observation so it's
actually just the posterior model
outputs
and you can see these shapes returning
in the state space
which in itself does not say that much
but it gives you at least
an kind of vague idea that at least the

state space is capturing something relevant for the problem i think there is one more animation on this yes so yeah it's the same conclusion as i said right now you can predict the future as you can see in the orange line and it appears to learn the velocity in its state space now if you can go to the next slides uh yeah um now we want to actually figure out how how we can use this model that we learned for active infants and typically in our audi would give some sparse reward for driving up the mountain but in our initial experiments we were more interested in how an agent could learn from human observation or demonstrations so we recorded seven i think yeah or five sorry five um human rollout in the environment just like driving around with the cards so you go first left and right and then we push these trajectories through our models together to our through our posterior model to get a preferred state distribution so here on this figure you can see the eight different dimensions how these evolve through time if you follow these trajectories so in the spread of course give gives you the standard deviation at each time point for

for that state value and then
if you use this these at this
preferred state distribution and you
calculate the g or this or even only the
kl divergence between your posterior and
your

um preferred distribution for different
trajectories through time

so you hear every different color every
different trajectory is a different
rollout in the environment or imaginary
rollout for the environment

you can see that you can actually use
this this prefer distribution to rank
trajectories according to uh

lowest free energy so you see that the
blue

curve is is the only one that reaches
the top

and is also the one that the model seems
to prefer

and then in the next slide yes

um we experiment a bit what this means
in terms of the actual free energy so
also compensated with the entropy

um and what we did here was we if
we had the agent observe the world for
one time step so we give it an initial
observation to get a

sort of bootstrap latent sample

and then we will only use the prior
model to imagine

what will happen if you follow certain
policies

so here we considered three policies or
like

uh two possible policies that you can
switch at three time steps

so you either go left right right left
right left at
etc or you can start with right and then
these are the blue curves
and what we see that if there is no
initial velocity
then um you see that the
agent imagines that the optimal policy
which is left right right
so first go as far up left and then just
blast your way to the top um
gives actually the left the least amount
of spread and also he believes that this
will
reach the top earlier than any other
policy
inversely if you look at the policies
that
first go right um then you see that
there is still a little and no doesn't
close to no spreads
due to the lack of initial velocity but
the agent already knows that
it's impossible to reach the top
now if you go to the next slides
uh the slide so here we do the exa the
same experiment
but we add some random initial velocity
and again we let the agent observe the
world for one time step
and then let plan according to the same
policies as before
and first of all you see that the agents
has a lot of
much larger spread on its believed
outcomes this is due to this
external extra uncertainty on
what the initial velocity is and also

this initial velocity might render
previously
infeasible policies feasible as you can
see at the bottom because
now if the agent thinks yes my initial
velocity is high enough
even with the one of the more
sub-optimal policies
it might still be possible for me to
reach the top
um and if you go to the next slide
please
here we can then see this in action so
this animation
we collapse the imagine trajectories to
the actual
trajectory and you see that
in the beginning if we can maybe play it
again in the beginning
it believes that more red policies will
reach the top and then
as it gains more momentum oh
it will believe that all blue policies
will reach the top and even it will know
that if i now
go left so i decelerate then i will less
likely
reach the top so i think yeah
so as you can see now he knows that red
will not reach the top
and yes
um then another experiment we did
building upon this was
what if we take this same approach but
now
we move to the task the kind of problems
we want to solve this so the high
dimensional

uh observations that you cannot model by
hands

so we took another opening agm
environments the

the car racer and here the goal is to
drive the red card as you can see on the
high resolution image on the right
left um on the road

for as long as you can and then we
trained our model on a handful of human
demonstrations of this and then you can
see that

in the reconstructed image on the right
it already it's even on a handful of
observations that can learn to predict
this

so we did exactly the same thing as
before and i think this is an animation
now

yeah so if you then use the exact
the the the planning tree that tim
explained on

this little card it will have learned to
that

the the road so the gray area is
important and it should stay on this
now the this agent is a bit more greedy
since it tries to

shortcut corners to get quicker on the
gray part as you can see here

okay if we can go to the next slide
maybe

so now and also an important point here
is that

our active entrance approach which is
model based

um seems to be a lot uh more data
efficient

it seems to be a lot more data efficient
than for example
uh a baseline rl agent so we took dqn
since this is also an off policy
algorithm
to be able to compare it kind of
similar to our approach and as you can
see in the
first graph so for the mounting car um
our model quickly um
so sorry our our models are in is the
orange
our mo quickly learns that at least some
method to reach the top and then
afterwards it just improves upon the
smiles
due to the sparseness of the rewards dqn
is not able to
learn as quickly and even after having
1000 times more thousand times more
observations itself
isn't capable of climbing the mountain
and even for the car racer it's even
worse
so here we trained on seven or ten
uh rollouts and immediately we're able
to get a reward of six hundreds
whilst dqn just fails to get that
same level of performance even after
thousands of
rollouts in the environment so if we can
go to the next slide
yeah and then a final experiment i want
to discuss today is a
the robotic navigation or like
maybe more accurately robotic control so
here we took
our kuka platform and we mounted some

sensors on top of it
and then also put on a laptop for good
measure to have some compute
um i don't think the sensors are really
relevant for what we're going to discuss
now so
then we drove around to the robot in our
lap as you can see on this movie just
like with the joystick and
captured a lot of data and we just drove
up and down the aisles
with robots now this environment is also
a bit challenging for robots since all
these aisles
are super similar like for a robot
knowing if it's an aisle one or two is
nearly the same thing it's very
difficult to comprehend for the
machine so then in this slide you see
what a recording might look like so you
have a rider a
lighter and radar feet and some images
yeah sorry tim this is now the correct
slide
and and then the goal is to
um again train a model that will be able
to generate
future observations for the robots i
don't know if there's an animation on
the slides
so yeah so you saw the first the high
resolution image was the real
observation
and this is then what the model thinks
will happen
if the robot first turns right and then
continues to drive
the little ghost artifact you saw a bit

a couple of frames ago so yeah we'll first look at it again so we drive and then it will turn and suddenly you will see a ghost them appear um this is because the model doesn't actually know what will happen it can only try to guess based on its previously learned experiences in the model and there's a lot of people walking around in the data set so there was a some chance that somebody would be walking there so it just might imagine that there's somebody there then maybe if we go to the next slides um this is also an animation so here you can see for example how these imaginary samples deviate and this is also the reason why we need this sampling and so the the different sampling and the planning tree four different outcomes so you see that given the same starting position and observation the model learns that turning left might have different outcomes depending where it is in there in which either this and we wear it in the air for example the top right then it the robot imagines that it is at the end of the aisle whilst in the bottom two it imagines that it is in the aisle so it just imagines some stuff on the racks

then here is and how we can evaluate the policies

so we typically we provided three possible policies turn left fourth and turn right

and then you can imagine what all these things will do in the environments and then similarly as before you can calculate the g and select the one that will most likely

bring you to your preferred sequences um

yeah right so also the nice thing of our model

is that you can

you can uh because of its because it's a neural network you can put

multiple types of observation into your posterior model you can fuse them

in various ways so what you can see here is that

similar to the camera feeds the robot will also learn the effect for example in a lighter scan and also it will learn the effects on the velocity bins in a radar

scan and this gives you of course extra robustness in your planning because you can

now reason on multiple modalities

then maybe in the next slides um

there are of course still some limitations to

using robotic to doing robotic control this way

first of all our robot is extremely short-sighted in time

it can only learn to predict as far

as the length of the sequences we
provided during training
and also the longer you roll out the
larger your
search tree becomes the more
computational
limits you will reach so
this is also then an area we are now
actively working on
um then currently our models still
require that we pre-record the data set
so
our models require that we drive around
ourselves and then
fit the model so there's also a point
we're
currently at this moment working on and
then
tying into this our models currently do
not really know how to explore
whilst there's probably a sensible way
to do exploration based
on the free energy principle since your
mobile uncertainty can be baked in
i think i don't know if you have any
more slides actually now
no desserts or maybe it's insane for
questions
awesome you can um unshare
and we can ask some questions so i
wrote down a bunch of stuff and also
anybody who's
watching live please ask some questions
so nice presentation though and
awesome very instructive videos
hopefully made us think made us laugh a
little bit when it cut
corners so maybe just a starting

question while people are
writing their question um what
brought you to be studying this topic
in this way were you coming from active
inference and saw
robotics as an interesting application
or
were you in the area of robotics and
then found active inference to be
a useful model
yeah so basically we were in the area of
um of robotics and reports of learning
and we were working on building
better low dimensional state
representations
to feed into a reinforcement learning
algorithm let's say that was our initial
idea
so just building representations
or for better reinforcement learning and
now we stumbled upon the
active inference framework which which
basically not only gave us a way
of
of how to uh to build degenerative
models because
basically we we found ourselves the
let's say the free energy of the that we
were basically already doing that
but we saw that it also gives us in the
same mathematical framework a way of
how to project these things to the
future and use these for planning
therefore resolving ambiguity and also
for
um scoring novelty and all these nice
properties that are basically lacking
in in rl so that's why we we basically

started
digging into this mathematical framework
to
go through all the the papers of call
and see how
all the ends are tied together and and
see whether this
would still work if you if you don't
have
your your state space defined the prompt
but you just learn it from
from data so that's how you start this
endeavor and that's
pretty much where we are at this point
and still investigating it further
awesome that was gonna be my second
question was like
what differences or advantages would you
describe for active inference
over reinforcement learning or other
machine learning frameworks
if you answered it in previously that's
great or do you want to add any other
thoughts yeah i think
i think the nice part is that

[Music]

you you automatically get the
the nice properties of of resolving
ambiguity and of
um potentially exploration if you also
um estimate posterior distributions over
your your parameters for example
so these are very interesting properties
uh mathematically however it's still
um it's there's still a gap to to
actually
um get this out of um
these real-world cases if you if you if

you don't have the degenerative model
uh predefined let's say so there are
still some challenges
but the theory at least this is is very
much appealing
um and i but i think if you look at
what's going on at the url side that we
refer to inside
then there's not that big of a gap
between
the two wheels i think because you get
all the curiosity bonuses that they try
to come up with
in in reinforcement learning and if you
look at the model based
stuff from like the niger hoffner with
this dreamer approach then everything is
kind of
similarly converging to it's a good idea
to build a model of your world
and get more sample efficiency and it
seems like a good idea to have some
planning in there
and maybe there is some and there's some
gaining
curiosity and so you see that a lot
in a lot of different independent
research tracks
we all converge along the same lines and
i think what's what is so appealing to
active inference is that basically
it brings this all together from from a
single principle
which makes it a a very nice framework
to
work with i think anything to add on
that ozone
oh yeah i was before tim mentioned that

i was already thinking along the lines
of
the work of the ninja however so i mean
if you look at his models and i think
that is currently
almost state of the art and mobile based
rl then
you'll find that the models they are
building are very similar to the models
we are building or other active
infrastructures are building
so yeah i mean it makes sense that all
these
approaches converge on a single id of
the idea works
yep very interesting that like plan to
dream and dream or imagine so that you
can sample
appropriately why put that as a second
layer on the model or have to
incentivize it
in a sort of ad hoc way why not have
that be the basis of the model so that's
a very nice
point so dean in the chat
asks have the authors heard of the
missionary and cannibals
game slash problem which is moving um
two kinds of mutually incompatible
agents back and forth from
two sides of a river and um if they have
heard of this or
thought of any kind of analogous cases
do you see any applications
yeah and i haven't heard of the game
before i think
it's probably similar to the like
crossing the river with a chicken the

fox and a
goat or what yes what was it yeah
there's like animal versions there's all
kinds of versions of this one but
going back and forth with different
kinds of incompatible agents and you
need to sort of
go to the left before you can make it up
the hill but it's a little bit of a
different setting so what does that make
you think of
i haven't actually really considered it
for our models but it might be
if we find a way to mod in an
environment and maybe
collect some data on it yeah i think
this is a problem that is very nicely
suited for
for doing it let's say the the vanilla
way where you basically
write out all the different states that
can happen
and kind of observe the outcomes that
you can have
so i i think you could actually
formalize it in such a way and then
run an active instrument simulation on
on that
and see and see what happens so
that's less of our interest because
in our cases we are mainly interested in
what if your
observations are so high dimensional
that you cannot even
start thinking about writing out
the the joint model let's say and the
only thing you can do
is interact with your environment and

try to

try to learn it from the data

which is a slightly different take on

on the active inference problem

great another question is

you are all deploying these models sort

of

real time with physical agents embodied

agents so what surprised you or what was

interesting to note

as far as going from the simulation only

where you can sort of put everything in

a box and know exactly what's going to

influence what

to the world of embodiment where i don't

know some dust could get into the robot

or

i saw a person walk by so what

comes into play when you actually deploy

physically and how does the model deal

with that

for me it's actually the thing i like

well first we

i played around a lot with the mountain

car and the car racer

but the first thing actually was a real

hurdle for me personally was

deploying it on a real robot you

suddenly have all these

uh hardware constraints like you can you

don't have infinite memory you don't

have server great compute anymore

and you all have to you have to yeah

fit it for example i think tim and i

spent a lot of time to make a demo

working

where we could roll this out real time

and

just the hardware constraints of doing
something as complex as active inference
real time
on a real power constraint robot is
another challenge
in and of itself yeah so i think
uh if if the question was what uh
what do you get from uh from doing it on
the real system well a lot of
frustration and pain i think is the
answer
but likewise if we then if it doesn't
works then the
gratitude is the satisfaction is so much
higher
so i i still remember ozone and me
cheering in the lab because we gave the
robot a preferred state of being like
nicely in the center of the ale
and it was actually moving along the ale
and then at the very end
they decided hmm this is not the center
of the animal and it just made a 360
degree turn
and started driving back and we were
like oh this is awesome
so i think the there's a lot of pain
frustration to get it work but then once
something comes out then then the
satisfaction is so much higher than
than when you see a mountain car which
reaches also
interesting you could set up a physical
valley
maybe make a physical mountain car
because there's so much comparison
of the software realization but
maybe that would be even taking it to

the next level
so another um thing that you talked
about repeatedly
maybe even in every example was actually
training the model
from just a handful of human cases like
you drove the car
you had people play the mountain car
game so
what exactly is happening there and how
does the model
like not over fit to the few
trajectories you show
or not just say hey you only gave me
three what's the deal
with these three totally different
trajectories like what exactly is being
learned or updated in the model when you
provide just a handful of human examples
i thought then we're going to answer
this one
yeah i think overfitting is is uh
is clearly an issue um
you you're no matter what you do you're
constrained kind of to the the data you
provide to the model so it cannot really
learn anything
else than than what it discovers so
that that's also what we pointed out at
the last slide that's a severe
limitation of our current
current work clearly but there is
um there is no other way when there is a
real robot involved
to get started say so that's the easier
way to get started so that's mainly the
driver but at the moment we're actually
working both in simulation but also on

real robots to see
can we actually get these systems to to
decide how to
um gather their own experience uh what
is
interesting to learn from because that's
actually one of the
to um shortcut to one of your previous
questions
what does active entrance give you well
that it actually can
can deal with these these kind of things
like my my robot needs to collect its
own experience what will you do that
would be always the same or
the explorer or this kind of thing so
these are really
some active areas of research but then
so to come back to
the overfitting problem so one thing
that
mitigates overfitting a bit is that the
the fact that you have all these
stochastics
even in the system it becomes sample
states
which makes it a bit more um but it
it's not like a classifier that that
always fits to
the test of let's say there's always
some kind of
noise in there but you're right um
it is in some sense overfitting the data
in the sense that it cannot predict
uh scenarios that it clearly hasn't seen
before
but so one of the one of the nice things
of having this uh

this expected energy formulation is that
you actually finish
um uh you're planning to
um with this entropy term which
basically means that
if you plan ahead in some space that
the the model wasn't trained on then
typically
you will have some more variation some
higher entropy
or that area because typically your your
observation becomes
very bad or um very blurry or
out of distribution and so so in some
sense the
the model is kind of robust against that
and it will also
if you then deploy a policy or like a
do the planning it will kind of try to
stay close to
where the model to the regime where the
model was restrained okay because there
you
basically get the the better predictions
so
uh in some sense that also mitigates the
the
problem also i want to add that
the by we we could use as
uh little of the uh rollouts as we did
because actually
if you had let a human do the rollout
you
solve the exploration problem for the
agent you already
get a very good coverage of the
relevant feasible state space you get
like for the car racer

if you let a random agent drive around
as exploration
99 of your day still just be grass and
the robot will
age will learn nothing about roads so by
having a human
human derive it you know okay this is
the road and
apparently this is important because
it's in every observation i've had
that almost makes me think of two ways
that we're seeing
these large models be trained with a
sort of mentor
like a human who says you know here's
the first places you want to be sampling
here's how you drive the first time
that's the driving instructor side by
side
and then there's the degenerative
adversarial approach
which is just the almost opposite like
we're going to be passing the model
the most confusing possible data
and so it's sort of like with this
carrot and the stick or the push in the
pole
these models from both of them maybe one
or maybe
both that they figure out how to be on
that razor's edge
and then in the race car example um
it was cutting corners so that just made
me wonder about
autonomous vehicles and you say okay
well the goal is to stay on the road
and to get there fast but then sometimes
get there fast

is gonna take priority and then all of a sudden you're way off the road and maybe now your car is ruined or something like that so if these were to be deployed like how will we even know what kind of preferences to instantiate the model with yeah that's a that's a very good point in fact because um how nice active entrance might look in in theory i think there it doesn't it's not a silver bullet to autonomous agents because a lot of the a lot of the civility is still in how do you how do you provide it with like the preferred prior distribution and that will be crucial in a real world system it's it's similar to a reward basically it's a bit more informative than just the scalar rewards you know let's say but it suffers from the same issues as in um if it's gets a way to do shortcuts um you get this preferred state that you didn't envision before as a as a designer of the of the experiment let's say it basically has the same issues as as reinforcements learning so i don't see it as as a silver bullet of solving autonomy um but at least it has some

um it has some um
burning knobs that you can you can you
can use to
at least avoid some cases like avoid
these ambiguity
ambiguous states or at least make it
first learn the model properly
so it has some nice properties but it's
not
a silver bullet to solving autonomous
systems i think
anything to add on that ozone i think
tim said it very well
yes the model is not a silver bullet
definitely
recognized what areas might be
interesting
shooting ranges first applications where
we can at least
explore it um are those the same use
cases that people
have been talking about just more
broadly
in terms of autonomous vehicles or might
there be
a sort of division of labor where active
inference is going to specialize like in
those
ambiguous scenarios or i just was
curious about that
okay so yeah i was i was also still
thinking but
like for us there i think the real next
application we're targeting
is just still the constrained navigation
and then for example a warehouse where
the impact of uh misplanning
is still fairly limited so actually more

realistic versions
of the situation we have in our lab
maybe it's also
the trajectory we're currently still on
so i think we still believe that there
is some value in
developing autonomous active entrance
agents
for this more industry-like settings
where you
can at least separate it the general
public
and more you right you have also some
level of control of the environment
yeah i think i i think you it's similar
to
to mitigating the same problem in your
learning is basically that you before
before you just let the system
randomly pick actions or pick any action
at once you kind of
or an environment where at least as a
human you can
you can shortcut the system and say okay
you want to drive forward into this rack
that's not a good idea so
you have at least some some ways of
defining some rules to keep it within
some safety range let's say
and within this boundary it can it can
for example
move autonomously but still then you you
might have the
nicer properties of the active inference
agent
that it if uh if you're driving around
in in some ales and somebody just
dropped dropped off the box in the

middle of the air
it will not freak out because the the
the slam map
is is uh it's no longer uh
consistent with with with how the robot
was programmed for example but we'll
just
say okay this is another case either it
experienced this before and it has in
his
in his world model an idea how to cope
with this
part will just be intrigued and start
learning on about this news information
so i think these are the nice properties
you have
in this agent and you kind of bypass the
all um um it's it's getting too greedy
to realize its preferences because you
kind of
shortcut these um these situations by by
having this
this based system in place that kind of
limits the the the
choice of actions in this game thanks
for the answer here's a
question from the chat as the code is
not disclosed
would you say sticking to what you write
in your paper
is sufficient to reproduce your results
or are there any further tricks you use
in designing and training the models
are you taking the art i'm sorry go
ahead tim first
yeah i think that together with the
appendices it should be
sufficient what do you think goes on

well i think

well we had some experience with that
where i tried to help somebody out who
was

trying to replicate our results um

i think them we had some tricks for the
planning

like that isn't as straightforward to
replicate but

the model prediction part

we explained the architectures and i uh
pretty detailed in the appendages so i
think

and we don't do any extra tricks for
example on on

data processing or our loss terms so
this should be easy to mimic but i think
the planning

is a bit more involved as we sometimes
ourselves have difficulties replicating
it

planning is hard yeah

um okay if anyone else has questions in
the live chat they can type that

um another piece that i thought was
really fascinating

in the mountain car example at least was
how you talked about

the noise allowing for the previously
implausible policies to become

possible like we saw a broader spread of
the

of the trajectories when there was noise

um how how is that being like

integrated in real time or how are the
noise

which are often very small how does that
change the model's understanding of

where it can go and what it should do
yeah so in the multi-car example there
are basically two sources of noise let's
say one is on
on the the noise on your observation
that you get so you get the
annoying estimate of your position and
so
this is typically not not so large
because yeah you need to have a
sufficient signal to noise ratio just to
learn anything let's say um
but the second part was
whether your agent starts with zero
velocity or with
a random velocity and these are
basically
two separately trained um models
so either you you have an agent always
starts with
a zero initial velocity and then
basically the model learns that
um the initial observation uh has
zero velocity and it's it basically
knows how to properly predict
from from the first observation on let's
say
if you train the model where the agent
constantly has
a random velocity then it basically
learns yeah from the first observation
still a lot of options can happen
depending on my velocity and
the more observations get in you see how
the model kind of
picks up okay this is now my velocity
and from then on
you see how the the wide range of

options collapses to
the most the most likely ones
anything on that or son well i was just
also thinking for example in the robotic
planning example we gave um
there you also see this spread but then
it's
more as in that the model learned that
at
different at similar state values
different outcomes are possible so it
will try to maybe
make the gaussian and the latent speeds
a bit wider so that if you sample from
it
you might get the slightly different
sample value
and that will then generate the
different outcome you want
you want to visualize so that's also so
even though the
standard normal you're sampling from
initially for your reparameterization
trick is
very as just standard normal the the
model can learn to inflate or deflate
this distribution so that you get a
wider coverage
for example you also saw in the in the
navigation example is that
basically the the model since we train
it on
on pretty short temporal subsequences of
like
i think one or two seconds in real time
um so it's not able to cause
to have a very consistent prediction for
for longer

times uh frames than that and also given
the fact that every ale
in the lab looks very similar it learns
the general structure like
there are wrecks left and right and
there might be boxes or
uh you saw that kind of the boxes or the
the stuff in the in this actually in the
racks is kind of very blurry and
it's kind of brownish uh blackish but
there's no real you don't really it
can identify what is in that wreck for
example and it's
so it has basically no spatial awareness
of
where in the wreck where in the air am i
am i at the very end or in the beginning
or in the middle
it has no idea that's why you saw if you
then say okay
turn right for a certain amount of time
and then it will either predict i'm in
the middle so the next
the if i turn 360 it will still be ale
after me
or i'm at the very end so if i turn
around i will see a wall
um so it it basically has no um
consistent knowledge of of where it is
and so the this is basically
model this kind of noise in the in
in distributions and if you if you draw
a different trajectory it will either
think
it's at the wall or it's facing the
other ale or there's a human passing by
so i see some
shady people-like structures so all

these kind of things are then modeled as as as kind of uh yeah noise in your in your distribution that just appears to be happening there makes sense um what might be helpful or required for long-term planning because the the tree that you had with a multiple um i guess bifurcations or whatever they represented that was very interesting how you had a very fully fleshed out tree and then you showed kind of how you recursed to prune back down to make a policy selection so as you suggested that exponentially explodes the computation what might be helpful or how can long-term planning be achieved with reasonable hardware yeah so so the the key thing here is hierarchical models i think and that's what we're uh we're pushing very hard on now is basically that you given your even a model like we trade now you basically put a new model on top of that that now as observations does not get uh does not get pixels but it gets state samples from this model and the the time step now is not to predict the next time step ahead but like to predict 10 times at the head or

20 or whatever or however you want to
course grain basically
and once you're at that point then you
have basically a system that can
can plan um if you plan ten time steps
you're actually planning 100 time steps
for the lower level model and so this
way you can keep on course grading that
you only have to explore
a few policies at each level and then
down below
again you only have to predict for like
one second that happens because
the other plan was made for by the
models on top
and so this way you can you can easily
uh bypass the and scale down the
complexity of the planning procedure
it reminds me a lot of driving where it
will be like okay
in five streets take a right turn so
you're not
pre-saging the right turn on okay one
two three four all right now i should
get ready
um what about symbolic information
like what if the aisle had a color
gradient or
if it had one dot two dot three dot
could that be learnt in an unsupervised
way um a symbol in that pixel level
model
or is that where a hierarchical model
would come into play
well i think the problem currently is
for us
architecture-wise our model isn't
capable of

capturing very like low level
details about environments just because
we are based on a va approach and then
the
mean squared error objective we use for
reconstruction and the wavy sample is
already inhibiting for example
recognizing dots
in your inputs um
maybe actually i don't know what thing
10 things about it i think color
gradients is something
like the model might learn if given
enough data and incentive
yeah so i think there's a number of
problems
with um with the approach in the sense
that
we're now doing prediction in in in
pixel space let's say
and the way you you calculate the
likelihoods
uh you evaluate the likelihood and you
you calculate the
reconstruction error it basically means
that
you want to have each pixel
independently predicted
on average quite right which means that
if you have very fine
grained details it easily um
ignores this you also have the
the complexity term that basically says
okay i want to have the
the least complex representation for for
reconstruction
for reconstruction but this basically
also means that

depending on on how much you you you put
pressure on
restricting the complexity the less
information you will actually encode and
the more blurrier basically your uh your
your your reconstruction will become so
it's very similar to
the bed of the ie uh to those behavior
where you have this better parameter
that
detunes how much weight you put on the
gal
divergence term versus the
reconstruction term
and this also has his impact on on what
the model will
actually put in the state representation
and and which details will
will be will be ignored
so yeah i think a lot of these things
are very difficult for the model
straight now just because the way we we
built
and parameterize the lightweight model
very interesting um how might somebody
go about learning or exploring this
like is there a textbook or the
citations that
are in your paper or hands-on what
would you encourage somebody who is
curious about this and wanted to
over the next maybe a few years be
following
yeah i mean we had a lot of like in the
active infant sports i think
when we started out there wasn't that
much information on how to do active
inference so we

had to figure it out the hard way by
trying a lot and failing a lot
but now currently i think even in this
model stream there was some
cool information and accessible
information on active inference
so and i mean specific implementation
wise
if you want to build a model
like this i think the current state of
the art in rl
and active inference i learned active
inference is
all pretty similar
yeah i i agree so
a lot of uh things have changed
regarding
how accessible the information on on
active influence has become
if you look now at a tutorial from ryan
smith for example which was also
extensively covered in one of your
videos
so i think that really it really knows
the the the entry
uh bar to just get to know the theory
and how everything
works or should work and to play around
with some
small toy examples and simulations that
you can get some insights on
on what does this thing do and how does
it work
for the deep learning part let's say to
build these models
then probably the the resources to go to
are just like
tutorials on on variational auto

encoders i think these are the
the things in deep learning that are
most
relevant for for for the active
inference work
we discuss here and if you can build a
variational auto encoder
and you know how active inference work
and you put the two together
together with some details from our
paper for example i think it should be
pretty straightforward to get to the
first working example cool
any closing thoughts or even
questions for our lab or
just to leave um for people to be
thinking about as they dream in
preparation for action
um yeah maybe i just want to
continue a bit on what tim said earlier
that
for example if you want to build
hierarchical models
you don't have to build an active
inference model on top of an active
inference model
we experimented some we did some
preliminary experiments on
for example just using our active
inference model as
dynamics model for a slam algorithm and
then you also already get
hierarchical modeling and
some of the long-term benefits so
maybe that's also interesting like to
think about is how this
active infrastructure fits in in already
existing techniques

yeah so i think what uh
what ozone wants to say is that in in
this case we use
like these deep neural nets to to
compare eyes
this transition model likely model and
posterior model but you shouldn't
always revert automatically to
these learning techniques so these are
kind of very popular right now and very
very cool but um in some cases you might
it might be sufficient if you know your
environment then
yeah you shouldn't bother learning the
state-space model if you know the state
space model just
yeah use it uh i know we used
the mountain car example here this would
be a good example of
uh why we shouldn't just use uh the deep
learning approach this was basically
just that like a proof of
principle for for us to get some simple
example working
but it only pays off i think if you have
like these real
high dimensional observations uh and you
don't
you have no clue how to characterize
your space-based mode but it's not that
this is like the
the default way to go let's say and
we're also like kind of
evolving especially uh also that if you
move to these hierarchical models
then we're kind of looking at how can we
make a more kind of discrete style
um paragraph state space model on top

where where we basically um
try to fit the whole um state space into
discrete
parts and then have like a simple
transition model like you
you gave the example yourself danielle
if you if you think about
navigating yourself you're not thinking
about predicting
all the pixels of all the houses uh or
but you just think about i i need to go
forward now and then the second street
to the left so you basically chunk up
the whole continuously state space into
some
relevant parts and then your transition
model
also becomes like a very simple
matrix with with transition
probabilities um
so i think the future is in erectile
models
and the future is in a mixture of um
some learned parts but also some um
some very discretized um intuitively
comprehensible parts just
two points on that um one is something
that's always drawn me to active
inference is that
it's inference conditioned and about
action
so you're not going for that 4k
google street view what does every house
look like
and again when that's the input data
even if you have a generative model
that's the output data
so active inference is a really

principled way to just sort of reduce
what you're predicting
to like which way should my elbow move
not
what will the pixels look like when my
elbow moves which may be just
taking gigabytes of data but if it's
just reduced to
the state space then it's easier to
learn and that's why
it was such an interesting contribution
with your paper
to actually learn that state space in
the context of
high resolution and real time and
heterogeneous
sensors and then also it's something
that we've seen
many perspectives on in the lab and in
these discussions
is there's the philosophical discussion
map and territory
who's really an active inference agent
is it everything a dust particle
a bacteria you know is the world built
this way
and then there's this sort of
engineering approach
where you have your preferences for how
you want to see the robot work
and then whatever you can tinker and
cobble together
that satisfies you and it reduced your
uncertainty about performing the task
you're trying to perform so it's sort of
sidesteps but then
it sidesteps those questions in a way
that actually brings us

to a higher level of understanding
because i know that
many listeners who are not as advanced
in the machine learning will be inspired
and have
qualitative thoughts based upon what you
brought here today
so thanks again for this awesome
presentation and conversation
and we'll always appreciate hearing any
follow-up whenever the time is right
yeah thank you daniel for having us
really nice discussion
okay peace see you later see ya bye
bye